

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn import svm
from sklearn.model_selection import train_test_split
```

```
In [2]: data = pd.read_csv('breastcancer.csv')
data.tail(10)
```

```
Out[2]:
```

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothn
--	----	-----------	-------------	--------------	----------------	-----------	---------

559	925291	B	11.51	23.93	74.52	403.5	
560	925292	B	14.05	27.15	91.38	600.4	
561	925311	B	11.20	29.37	70.67	386.0	
562	925622	M	15.22	30.62	103.40	716.9	
563	926125	M	20.92	25.09	143.00	1347.0	
564	926424	M	21.56	22.39	142.00	1479.0	
565	926682	M	20.13	28.25	131.20	1261.0	
566	926954	M	16.60	28.08	108.30	858.1	
567	927241	M	20.60	29.33	140.10	1265.0	
568	92751	B	7.76	24.54	47.92	181.0	

10 rows × 33 columns



Cleaning the Dataset

```
In [3]: data.dropna()
data.isnull().sum()
```

```

Out[3]: id                0
        diagnosis         0
        radius_mean       0
        texture_mean      0
        perimeter_mean    0
        area_mean         0
        smoothness_mean   0
        compactness_mean  0
        concavity_mean    0
        concave points_mean 0
        symmetry_mean     0
        fractal_dimension_mean 0
        radius_se         0
        texture_se        0
        perimeter_se      0
        area_se           0
        smoothness_se     0
        compactness_se    0
        concavity_se      0
        concave points_se 0
        symmetry_se       0
        fractal_dimension_se 0
        radius_worst      0
        texture_worst     0
        perimeter_worst   0
        area_worst        0
        smoothness_worst  0
        compactness_worst 0
        concavity_worst   0
        concave points_worst 0
        symmetry_worst    0
        fractal_dimension_worst 0
        Unnamed: 32       569
        dtype: int64

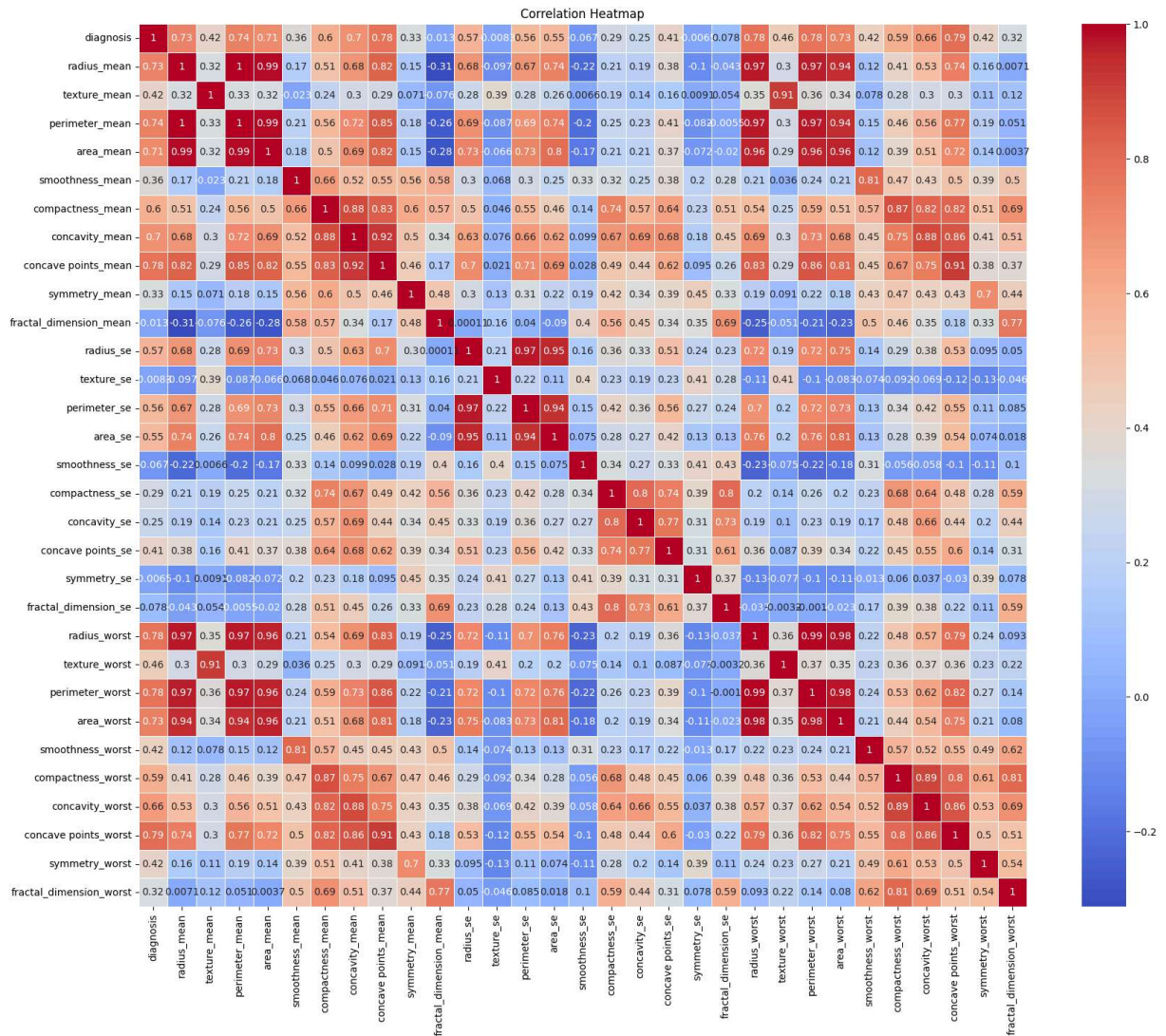
```

Making the correlation heatmap

```

In [4]: data_clean = data.drop(['id', 'Unnamed: 32'], axis=1) # drop unnecessary columns
        data_clean['diagnosis'] = data_clean['diagnosis'].map({'B': 0, 'M': 1})
        corr_matrix = data_clean.corr()
        plt.figure(figsize=(20, 16))
        sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', linewidths=0.5)
        plt.title('Correlation Heatmap')
        plt.show()

```



Conclusion: The features that we will be using for diagnosis are:

- radius_mean - 0.73
- concavity_mean - 0.7
- texture_mean - 0.42
- compactness_mean - 0.6
- smoothness_mean - 0.36
- symmetry_mean - 0.33

```
In [5]: x = data[["radius_mean","texture_mean","smoothness_mean","compactness_mean","concav
y = data["diagnosis"]
```

Splitting for testing and training

```
In [6]: x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_sta
```

```
In [7]: from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(x_train)
X_test_scaled = scaler.transform(x_test)
```

```
In [8]: from sklearn.svm import SVC

model = SVC(kernel='linear')
model.fit(X_train_scaled, y_train)
```

```
Out[8]: SVC
SVC(kernel='linear')
```

```
In [9]: from sklearn.metrics import classification_report, confusion_matrix, accuracy_score

y_pred = model.predict(X_test_scaled)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

Accuracy: 0.9473684210526315

Classification Report:

	precision	recall	f1-score	support
B	0.95	0.97	0.96	71
M	0.95	0.91	0.93	43
accuracy			0.95	114
macro avg	0.95	0.94	0.94	114
weighted avg	0.95	0.95	0.95	114

Confusion Matrix:

```
[[69  2]
 [ 4 39]]
```

we get a 95% accuracy

Plotting the hyperplane

```
In [10]: import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.svm import SVC

#Feature selection
features = ['radius_mean', 'concavity_mean', 'texture_mean', 'compactness_mean', 's
X = data[features]
y = data['diagnosis'].map({'B': 0, 'M': 1}) # Encode Labels
```

```

# Scaling
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

#PCA for 2D visualization
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

# Train Linear SVM
clf = SVC(kernel='linear')
clf.fit(X_pca, y)

# Plot
plt.figure(figsize=(10, 7))

# Plot decision regions
ax = plt.gca()
xlim = ax.get_xlim()
ylim = ax.get_ylim()

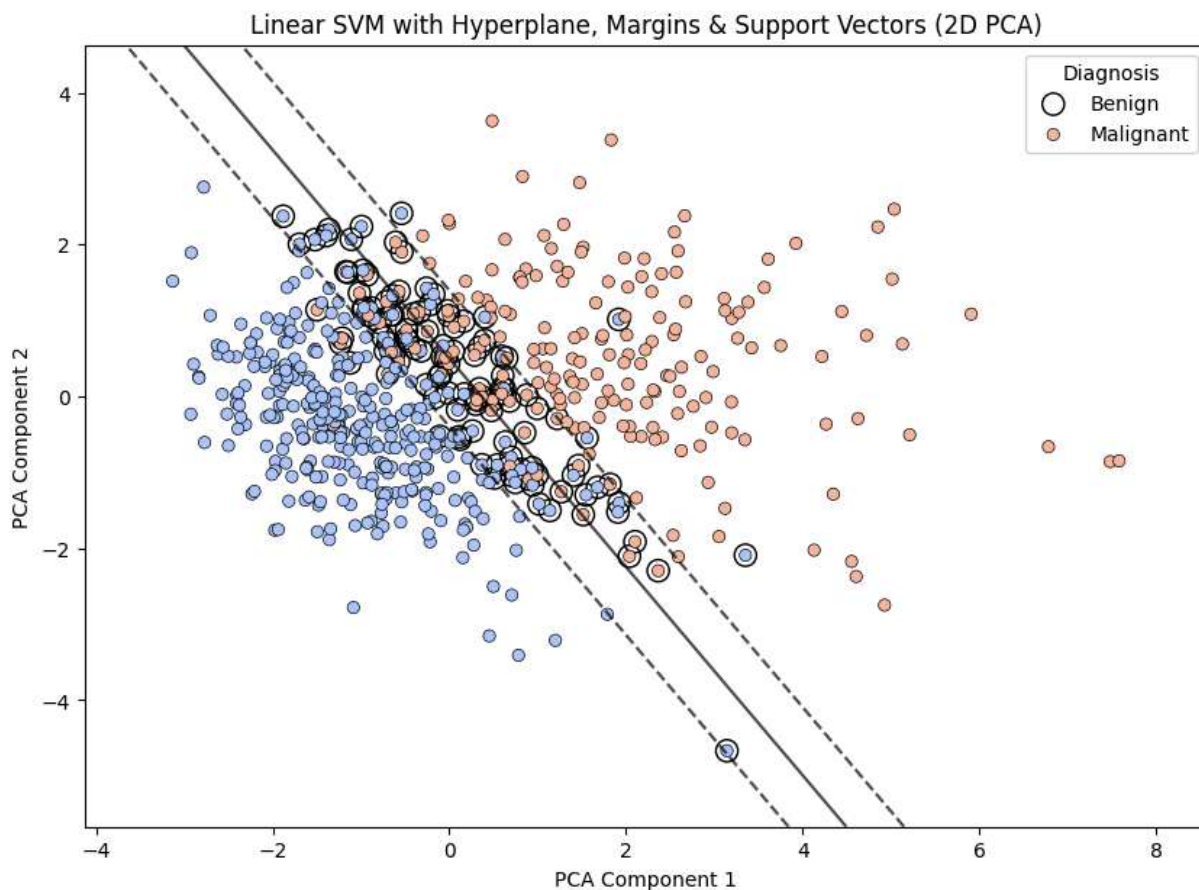
# Create grid to evaluate model
xx = np.linspace(X_pca[:, 0].min() - 1, X_pca[:, 0].max() + 1, 500)
yy = np.linspace(X_pca[:, 1].min() - 1, X_pca[:, 1].max() + 1, 500)
YY, XX = np.meshgrid(yy, xx)
xy = np.vstack([XX.ravel(), YY.ravel()]).T
Z = clf.decision_function(xy).reshape(XX.shape)

# Plot decision boundary and margins
contour = plt.contour(XX, YY, Z, colors='k',
                      levels=[-1, 0, 1], alpha=0.7,
                      linestyles=['--', '-', '--'])

# Plot support vectors
plt.scatter(clf.support_vectors_[:, 0], clf.support_vectors_[:, 1],
           s=120, facecolors='none', edgecolors='k', label='Support Vectors')

# Plot the data points
sns.scatterplot(x=X_pca[:, 0], y=X_pca[:, 1], hue=y, palette='coolwarm', edgecolor=
plt.title("Linear SVM with Hyperplane, Margins & Support Vectors (2D PCA)")
plt.xlabel("PCA Component 1")
plt.ylabel("PCA Component 2")
plt.legend(title='Diagnosis', labels=['Benign', 'Malignant'])
plt.show()

```



Non Linear SVM using RBF kernel

```
In [11]: from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, confusion_matrix

#Select features & Label
features = ['radius_mean', 'concavity_mean', 'texture_mean', 'compactness_mean', 's
X = data[features]
y = data['diagnosis'].map({'B': 0, 'M': 1})

#Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_sta

# Scale
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

#Train non-linear SVM with RBF kernel
clf_rbf = SVC(kernel='rbf', C=1, gamma='scale')
clf_rbf.fit(X_train_scaled, y_train)

#Evaluate
y_pred = clf_rbf.predict(X_test_scaled)
```

```
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

Confusion Matrix:

```
[[71  0]
 [ 3 40]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.96	1.00	0.98	71
1	1.00	0.93	0.96	43
accuracy			0.97	114
macro avg	0.98	0.97	0.97	114
weighted avg	0.97	0.97	0.97	114

We get 97% accuracy